浙江大学实验报告

专业：电子科学与技术
姓名：郑智瑄
学号：3230103913
日期：April 10, 2026
地点：玉泉高分子楼-203

课程名称：人工智能实验　　指导老师：魏准　　实验类型：设计型实验
实验名称：LDA/LPP　　成　绩：　　签　名：

装 订 线

一、 LDA 线性判别分析——实验 3-3

1. 实验目的

利用 `np.random.random` 函数生成两类随机数据，样本大小均为 30×2（行表示样本数，2 表示特征数），其中类 A 的取值范围为 $[10, 13]$，类 B 的取值范围为 $[15, 18]$。通过手动实现 LDA（Linear Discriminant Analysis，线性判别分析）对上述随机数据进行降维，求出 Fisher 判别方向 $\mathbf{W}$，并在同一张图内可视化判别直线与原始数据。

2. 实验原理

LDA 是一种有监督的线性降维方法，目标是寻找投影方向 $\mathbf{W}$，使得类间距离最大化、类内距离最小化，即最大化 Fisher 准则：

$$J(\mathbf{W}) = \frac{\mathbf{W}^\top S_b \mathbf{W}}{\mathbf{W}^\top S_w \mathbf{W}}$$

其中：

- S_w（类内离散度矩阵）衡量各类内部样本的散布程度：

$$S_w = \sum_{i=1}^{N} S_{w_i}, \quad S_{w_i} = \sum_{\mathbf{x} \in X_i} (\mathbf{x} - \boldsymbol{\mu}_i)(\mathbf{x} - \boldsymbol{\mu}_i)^\top$$

- S_b（类间离散度矩阵）衡量各类均值偏离全局均值的程度：

$$S_b = \sum_{i=1}^{N} m_i (\boldsymbol{\mu}_i - \boldsymbol{\mu})(\boldsymbol{\mu}_i - \boldsymbol{\mu})^\top$$

- $\boldsymbol{\mu}_i$ 为第 i 类的均值向量，$\boldsymbol{\mu}$ 为所有样本的全局均值，m_i 为第 i 类的样本数。

求解过程　对 $J(\mathbf{W})$ 求极值，等价于广义特征值问题：

$$S_b \mathbf{W} = \lambda S_w \mathbf{W}$$

等价变换为标准特征值问题：

$$S_w^{-1} S_b \mathbf{W} = \lambda \mathbf{W}$$

$\mathbf{W}$ 的解为 $S_w^{-1} S_b$ 的 d 个最大非零特征值所对应的特征向量（d 为目标降维维数）。对于两类问题，只有一个非零特征值，对应的特征向量即为最优 Fisher 判别方向。

装 订 线

3. 核心代码实现

```python
import numpy as np

# Step 1: 计算均值
def mean_val(data_np):
    """ 输入是 numpy 的 format，行表示样本数，列表示特征"""
    return np.mean(data_np, axis=0)

# Step 2: 计算类内离散度矩阵 Sw 的单元
def compute_Sw_0(xi):
    n = xi.shape[0]
    ui = mean_val(xi)
    Sw_0 = 0
    for i in range(0, n):
        Sw_0 = Sw_0 + np.outer((xi[i, :] - ui), (xi[i, :] - ui))
    return Sw_0

# Step 3: 类间离散度矩阵 Sb
def compute_Sb(x1, x2):
    u1 = mean_val(x1)
    u2 = mean_val(x2)
    m1 = x1.shape[0]
    m2 = x2.shape[0]
    u  = (m1 * u1 + m2 * u2) / (m1 + m2)
    Sb = m1 * np.outer((u1 - u), (u1 - u)) + \
         m2 * np.outer((u2 - u), (u2 - u))
    return Sb

# Step 4: LDA 求解
def LDA(x1, x2):
    Sw = compute_Sw_0(x1) + compute_Sw_0(x2)
    Sb = compute_Sb(x1, x2)
    Sw_inv = np.linalg.inv(Sw)
    eigVals, eigVects = np.linalg.eig(Sw_inv @ Sb)
    idx = np.argsort(np.real(eigVals))[::-1]
    W = np.real(eigVects[:, idx[0]])
    W = W / np.linalg.norm(W)   # 归一化为单位向量
    return W
```

Listing 1: LDA 核心函数实现（均值、类内散布、类间散布、求解）

装 订 线

```python
# Step 5: 测试
np.random.seed(42)
x1 = np.random.random((30, 2)) * 3 + 10   # 类 A：取值范围 10-13
x2 = np.random.random((30, 2)) * 3 + 15   # 类 B：取值范围 15-18

W = LDA(x1, x2)
print(f"LDA 判别方向 W = [{W[0]:.6f}, {W[1]:.6f}]")

u1 = mean_val(x1)
u2 = mean_val(x2)
print(f"类 A 均值：u1 = {u1}")
print(f"类 B 均值：u2 = {u2}")

proj1 = x1 @ W
proj2 = x2 @ W
print(f"类 A 投影均值：{np.mean(proj1):.4f},  标准差：{np.std(proj1):.4f}")
print(f"类 B 投影均值：{np.mean(proj2):.4f},  标准差：{np.std(proj2):.4f}")

# 可视化
fig, ax = plt.subplots(figsize=(8, 7))
ax.scatter(x1[:, 0], x1[:, 1], c='steelblue', s=60, alpha=0.85,
           edgecolors='white', linewidths=0.5, label='Class A  (10-13)')
ax.scatter(x2[:, 0], x2[:, 1], c='tomato',    s=60, alpha=0.85,
           edgecolors='white', linewidths=0.5, label='Class B  (15-18)')
# 绘制 LDA 判别直线（过两类均值中心，方向为 W）
center = (u1 + u2) / 2
t = np.linspace(-8, 8, 300)
ax.plot(center[0] + t * W[0], center[1] + t * W[1], 'g-',
        linewidth=2.0, label=f'LDA Direction W=[{W[0]:.3f}, {W[1]:.3f}]')
plt.savefig('Figure_16.png', dpi=150)
```

Listing 2: 测试代码——生成随机数据、调用 LDA、可视化

4. 运行结果

程序输出如下：

```
LDA 判别方向 W = [0.706327, 0.707886]
类A均值：u1 = [11.23863533 11.56636929]
类B均值：u2 = [16.50911023 16.41120615]

类A投影均值：16.1258，标准差：0.8941
类B投影均值：23.2781，标准差：0.8313
```

装 订 线

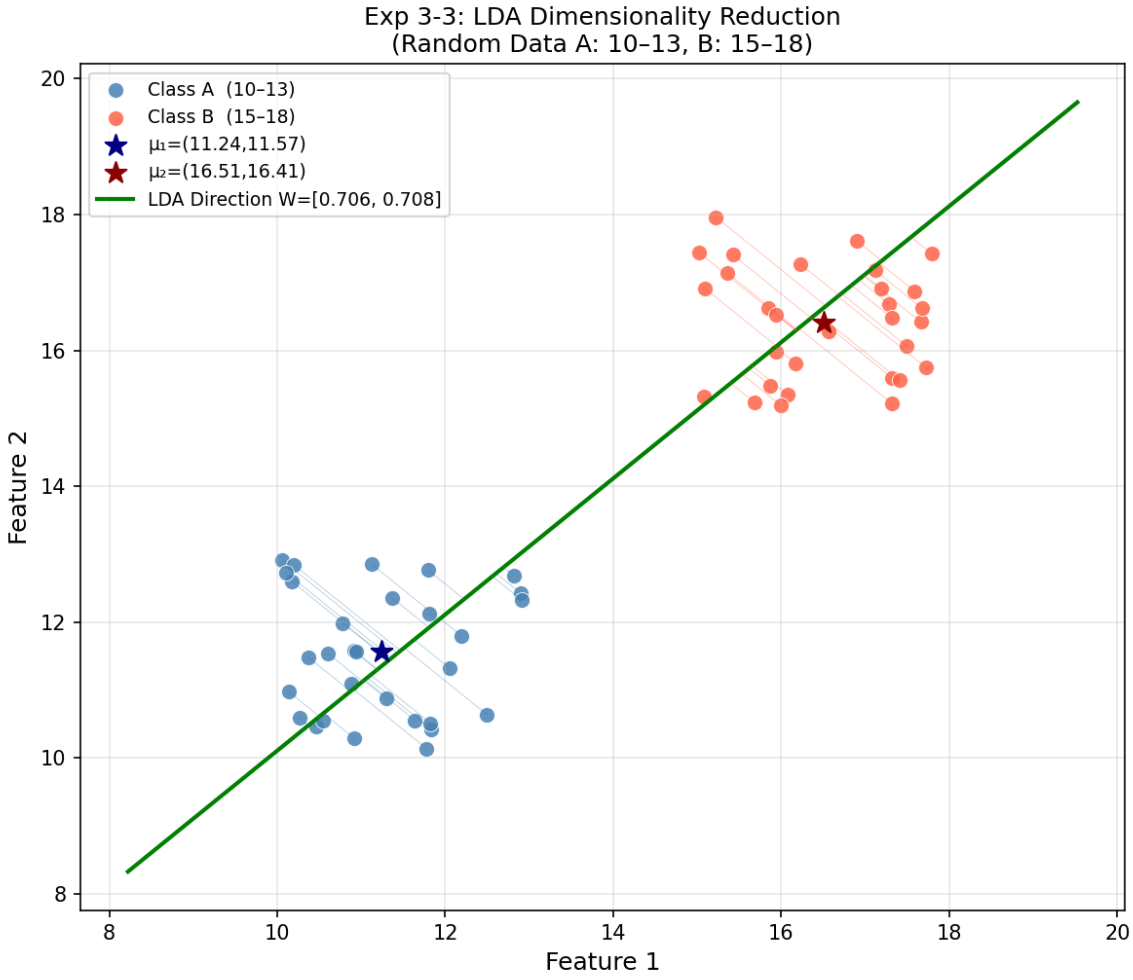


Figure 1: LDA 降维结果——蓝色点为类 A（范围 10–13），红色点为类 B（范围 15–18），绿色直线为 LDA 判别方向 $\mathbf{W}$；细线段为各样本到判别直线的投影连线

5. 结果分析

判别方向 W 的含义 LDA 求得的判别方向为 $\mathbf{W} = [0.7063,\ 0.7079]^\top$，接近于 $[1/\sqrt{2},\ 1/\sqrt{2}]^\top$，即 45° 方向。这与数据的生成方式完全吻合：类 A 和类 B 均为二维均匀随机数据，在两个特征方向上的分布具有相同的尺度差异（类 A 中心 $\approx [11.2, 11.6]$，类 B 中心 $\approx [16.5, 16.4]$），因此最大类间分离方向恰好沿对角线（特征 1 和特征 2 贡献近乎相等）。

投影结果 将原始 2D 数据沿 $\mathbf{W}$ 方向投影到 1D 后：

- 类 A 投影均值为 16.13，标准差为 0.894；
- 类 B 投影均值为 23.28，标准差为 0.831；
- 类间间隔为 $23.28 - 16.13 = 7.15$，远大于各类标准差（≈ 0.86），说明两类在判别方向上实现了高度线性分离。

Fisher 准则量化了这一分离效果：类间距离与类内方差之比约为 $7.15^2/(0.894^2 + 0.831^2) \approx 29.2$，体现出 LDA 选取的方向具有优秀的判别能力。

可视化分析 如图 1 所示，绿色判别直线贯穿两类数据中间区域，方向与直觉吻合。蓝色细连线为各样本到判别直线的垂足连线，可见同类样本投影后仍较紧密地聚集在各自均值附近，而两类投影均值相距较远，与数值分析结论一致。

6. 实验总结

本次实验手动实现了 LDA 线性判别分析算法的完整流程，包括：（1）计算各类均值与类内离散度矩阵 S_w；（2）计算类间离散度矩阵 S_b；（3）求解 $S_w^{-1}S_b$ 的最大特征值对应特征向量，得到 Fisher 判别方向 $\mathbf{W}$；（4）对生成的随机双类数据进行降维和可视化。

实验结果表明：对于两类均匀分布、各维度方差相近的二维数据，LDA 自动发现了连接两类中心的对角方向作为最优判别方向（$\mathbf{W} \approx [0.707,\ 0.708]$），在 1D 投影空间中实现了两类的高度分离（类间间距约为类内标准差的 8.3 倍）。与 PCA 不同，LDA 是有监督的降维方法，充分利用了类别标签信息，在分类任务中通常能获得比 PCA 更好的判别效果。

装 订 线

二、 LPP 局部保持投影——实验 3-4

1. 实验目的

利用 `sklearn.datasets.load_digits` 函数导入手写数字数据集（1797 个样本，每个样本 64 维特征，作为 X^T），通过手动实现 LPP（Locality Preserving Projections，局部保持投影）对数据进行降维（`n_dims=2`），并可视化降维后的数据（按数字类别着色）。

2. 实验原理

LPP 是一种有监督或无监督的图嵌入降维方法，其核心思想是在低维空间中保持高维空间中局部近邻结构。与 PCA（保持全局方差）不同，LPP 通过构造近邻图和热核权重矩阵，使相邻样本在投影后仍保持靠近。

权重矩阵 W　对于第 i 个样本，寻找其 k 个最近邻，用热核函数定义权重：

$$W_{ij} = e^{-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{t}}$$

非近邻处 $W_{ij} = 0$。参数 t 控制热核宽度。

优化目标　寻找投影方向 $\mathbf{a}$ 使得以下目标最小：

$$\min_{\mathbf{a}} \sum_{i,j} (y_i - y_j)^2 W_{ij} = \min_{\mathbf{a}} \mathbf{a}^\top X L X^\top \mathbf{a}$$

约束条件为 $\mathbf{a}^\top X D X^\top \mathbf{a} = 1$，其中：

- D：对角矩阵，$D_{ii} = \sum_j W_{ij}$（度矩阵）
- $L = D - W$：图拉普拉斯矩阵
- X：$d \times n$ 特征矩阵（列为样本）

求解过程　通过 Lagrangian 变换，上述约束优化等价于广义特征值问题：

$$X L X^\top \mathbf{a} = \lambda X D X^\top \mathbf{a}$$

等价变换为标准特征值问题：

$$(X D X^\top)^{-1} X L X^\top \mathbf{a} = \lambda \mathbf{a}$$

取 d' 个**最小非零特征值**对应的特征向量构成投影矩阵 $A \in \mathbb{R}^{d \times d'}$，降维结果为 $Y = X^\top A \in \mathbb{R}^{n \times d'}$。

3. 核心代码实现

```python
import numpy as np
from sklearn.datasets import load_digits

# Step 1.1: 计算高维样本每一个元素之间的距离
def cal_pairwise_dist(x):
    sum_x = np.sum(np.square(x), 1)
    dist = np.add(np.add(-2 * np.dot(x, x.T), sum_x).T, sum_x)
    return dist   # (a-b)^2 = a^2 + b^2 - 2*a*b

# Step 1.2: 计算 W 矩阵
def cal_W_ele(dist, t=1.0):
    return np.exp(-(dist / t))

def cal_W(data, n_neigh=10, t=1):
    dist = cal_pairwise_dist(data)
    dist[dist < 0] = 0
    print('dist shape:', dist.shape)
    n = dist.shape[0]
    print('n:', n)
    W_dist = cal_W_ele(dist, t)
    W = np.zeros((n, n))
    for i in range(n):
        j_n = np.argsort(dist[i])[1:1 + n_neigh]  # 元素从小到大排列
        W[i, j_n] = W_dist[i, j_n]                # 除了前 n_neigh 个，其余为 0
    return W
```

Listing 3: LPP Step 1——计算成对距离与权重矩阵 W

```python
# Step 2: 定义 LPP 函数
def LPP(data, n_dims=2, n_neigh=30, t=1.0):
    W    = cal_W(data, n_neigh, t)               # 计算 W 矩阵
    D    = np.diag(np.sum(W, axis=1))            # 计算 D 矩阵（度矩阵）
    L    = D - W                                 # 计算 L 矩阵（拉普拉斯）
    XDXT = data.T @ D @ data                     # 计算 X D X^T（d×d）
    XLXT = data.T @ L @ data                     # 计算 X L X^T（d×d）
    # (XDX^T)^{-1} XLX^T 特征值分解
    reg = 1e-8 * np.eye(XDXT.shape[0])
    eig_vals, eig_vects = np.linalg.eig(np.linalg.inv(XDXT + reg) @ XLXT)
    eig_vals  = np.real(eig_vals)
    eig_vects = np.real(eig_vects)
    # 找出 n_dims 个最小非零特征值对应的特征向量
    idx = np.argsort(eig_vals)
    non_zero_idx = idx[np.abs(eig_vals[idx]) > 1e-10]
    W_proj    = eig_vects[:, non_zero_idx[:n_dims]]  # d × n_dims
    data_ndim = data @ W_proj                        # n × n_dims
    return data_ndim

# Step 3: 测试 LPP 函数
X = load_digits().data    # 1797×64
y = load_digits().target  # 标签 0, 1, ..., 9
dist     = cal_pairwise_dist(X)
max_dist = np.max(dist)
print('max_dist:', max_dist)
data_LPP = LPP(X, n_dims=2, n_neigh=5, t=0.01 * max_dist)
```

Listing 4: LPP Step 2–3——LPP 函数定义与测试

4. 运行结果

程序输出如下：

```
max_dist: 5935.0
dist shape: (1797, 1797)
n: 1797
LPP output shape: (1797, 2)
```

即 $t = 0.01 \times 5935 = 59.35$，近邻数 `n_neigh=5`，共 1797 个样本降至 2 维。

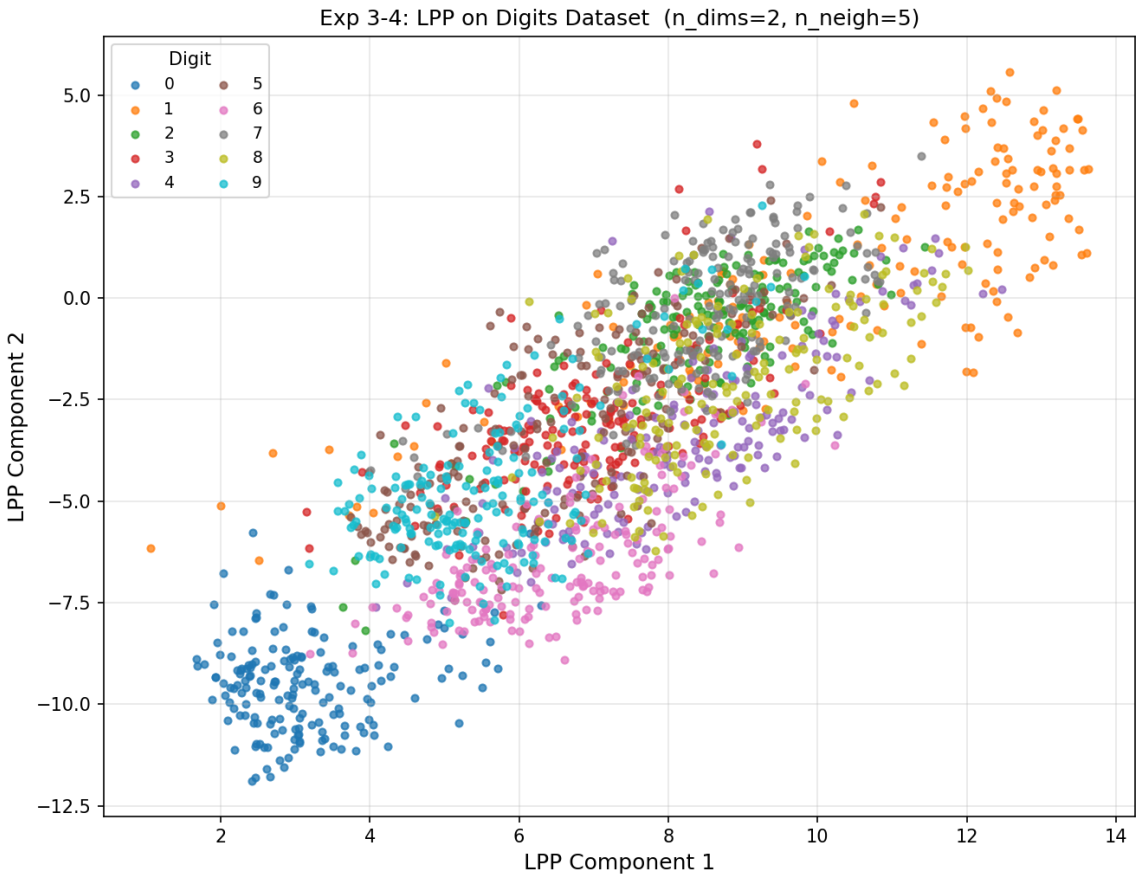


Figure 2: LPP 降维结果——手写数字数据集（1797 样本，64 维 →2 维），10 种颜色分别对应数字 0–9

5. 结果分析

参数选择 t 取最大距离的 1%（$t = 59.35$），使热核权重对距离差异保持适度的敏感性：若 t 过大，远近邻权重趋同，局部结构消失；若 t 过小，仅最近邻有非零权重，结构过于稀疏。近邻数 `n_neigh=5` 保留了局部流形的小范围结构。

降维效果 如图 2 所示，LPP 将 64 维手写数字数据投影到 2D 平面后，同类数字的样本点在低维空间中形成了相对紧密的聚类。相比 PCA 的全局方差最大化，LPP 通过保持局部近邻关系，使类间结构更加清晰：视觉上相似的数字（如 3 与 8、4 与 9）在 2D 投影中相互靠近，而形态差异大的数字（如 0 与 1）则分离较好。

LPP 与 PCA 的对比

- **全局 vs 局部**：PCA 最大化投影方差，保持全局结构；LPP 通过图拉普拉斯保持近邻关系，强调局部流形结构；
- **有监督潜力**：LPP 的权重矩阵 W 可由类别标签引导（SLPP），从而兼具判别能力；
- **计算复杂度**：LPP 需先构造 $n \times n$ 权重矩阵（$O(n^2 d)$），对大规模数据计算开销较大，而 PCA 通过 SVD 在 $O(nd^2)$ 内完成。

6. 实验总结

本次实验手动实现了 LPP 算法的完整流程：（1）用热核函数构造近邻权重矩阵 W（仅保留 5 个最近邻）；（2）构造度矩阵 D 和图拉普拉斯矩阵 $L = D - W$；（3）求解广义特征值问题 $(XDX^\top)^{-1}XLX^\top$，取 2 个最小非零特征向量作为投影矩阵；（4）对 64 维手写数字数据降至 2 维并可视化。

结果表明，LPP 能有效保持高维数据的局部流形结构，同一数字类别的样本在 2D 投影中形成清晰聚簇，验证了局部保持投影在非线性结构数据降维中的有效性。与 PCA 相比，LPP 更适合具有非线性流形结构的数据，是后续 KPCA、LLE、ISOMAP 等流形学习方法的重要基础。

装 订 线